Sesión 6 — Laboratorio

Programación Declarativa
3º del Grado en Ingeniería Informática
Funciones monomórficas, polimórficas e inferencia de tipos

Nota

Abre GHCi y carga tus definiciones con `:load fichero.hs` (o `:l fichero.hs`). Recuerda que puedes recargar tras editar con `:reload` (o `:r`). Comprueba los tipos de tus funciones con `:type` (`:t`) antes de probarlas.
Los ejercicios marcados con ★ son los recomendados para completar durante la sesión si el tiempo es limitado.

Parte A — Funciones monomórficas

Ejercicio 1.

Define `areaCirculo :: Float -> Float` que, dado el radio, devuelva el área del círculo ($A = \pi r^2$).

```
ghci> areaCirculo 1
3.1415927
ghci> areaCirculo 2
12.566371
```

Pista: la constante π está disponible en Haskell como `pi`.

Ejercicio 2. ★

Define `esBisiesto :: Int -> Bool` usando guardas. Un año es bisiesto si es divisible entre 4, salvo que sea divisible entre 100 y no entre 400.

```
ghci> esBisiesto 2024
True
ghci> esBisiesto 1900
False
ghci> esBisiesto 2000
True
```

Ejercicio 3.

Define `celsiusAFahrenheit :: Float -> Float` que convierta grados Celsius a Fahrenheit mediante la fórmula $F = \frac{9}{5} \cdot C + 32$.

```
ghci> celsiusAFahrenheit 0
32.0
ghci> celsiusAFahrenheit 100
212.0
ghci> celsiusAFahrenheit (-40)
-40.0
```

Ejercicio 4.

Define `maxDeTres :: Int -> Int -> Int -> Int` que devuelva el máximo de tres enteros. Puedes usar `if-then-else`, `case` o guardas: elige la opción que te parezca más clara.

```
ghci> maxDeTres 3 7 5
7
ghci> maxDeTres 1 1 1
1
```

Ejercicio 5.

Define `raicesCuadradas :: Float -> Float -> Float -> (Float,Float)` que, dados los coeficientes a, b, c de $ax^2 + bx + c = 0$, devuelva el par de raíces, usando una cláusula `where` para no repetir cálculos. Supón que el discriminante es siempre no negativo.

```
ghci> raicesCuadradas 1 (-3) 2
(2.0,1.0)
```

Ejercicio 6.

Define `clasificaIMC :: Float -> Float -> String` que, dados el peso (kg) y la altura (m), clasifique el IMC ($imc = peso/altura^2$) usando guardas: `"bajo peso"` si `imc < 18.5`, `"normal"` si `imc < 25.0`, `"sobrepeso"` si `imc < 30.0`, y `"obesidad"` en otro caso.

```
ghci> clasificaIMC 70 1.75
"normal"
```

Parte B — Funciones polimórficas

Ejercicio 7.

Define `primero3 :: [a] -> [a]` que devuelva los 3 primeros elementos de una lista (o todos, si tiene menos de 3), usando `take`.

```
ghci> primero3 [1,2,3,4,5]
[1,2,3]
ghci> primero3 "ab"
"ab"
```

Ejercicio 8.

Define `ultimoElemento :: [a] -> a` que devuelva el último elemento de una lista no vacía, usando `last`.

```
ghci> ultimoElemento [1,2,3]
3
ghci> ultimoElemento "hola"
'a'
```

¿Qué ocurre si se llama a `ultimoElemento []`? Pruébalo en GHCi.

Ejercicio 9.

Define `intercambiarPar :: (a,b) -> (b,a)` que invierta los componentes de una tupla de dos elementos.

```
ghci> intercambiarPar (1,'a')
('a',1)
ghci> intercambiarPar ("hola",True)
(True,"hola")
```

Ejercicio 10. ★

Define `aplicarDosVeces :: (a -> a) -> a -> a` que aplique una función `f` dos veces a su argumento.

```
ghci> aplicarDosVeces (+3) 10
16
ghci> aplicarDosVeces reverse [1,2,3]
[1,2,3]
ghci> aplicarDosVeces tail "Haskell"
"skell"
```

¿Qué tipo infiere GHCi para `aplicarDosVeces aplicarDosVeces`?

Ejercicio 11. ★

Define `contarOcurrencias :: Eq a => a -> [a] -> Int` que cuente cuántas veces aparece un valor en una lista, usando `length` y `filter`.

```
ghci> contarOcurrencias 'l' "Hello, world!"
3
ghci> contarOcurrencias 5 [1,5,2,5,5]
3
```

Ejercicio 12.

Define `eliminaTodas :: Eq a => a -> [a] -> [a]` que elimine **todas** las apariciones de un valor en una lista, usando `filter`.

```
ghci> eliminaTodas 2 [1,2,3,2,4]
[1,3,4]
ghci> eliminaTodas 'a' "banana"
"bnn"
```

Parte C — `map`, `filter` y secciones

Ejercicio 13. ★

Define `cuadradosPares :: [Int] -> [Int]` que devuelva los cuadrados de los números pares de una lista, combinando `map` y `filter` con secciones (sin usar listas por comprensión).

```
ghci> cuadradosPares [1,2,3,4,5,6]
[4,16,36]
```

Ejercicio 14.

Define `tablaMultiplicar :: Int -> [Int]` que devuelva la tabla de multiplicar de un número del 1 al 10, usando `map` con una sección sobre `[1..10]`.

```
ghci> tablaMultiplicar 7
[7,14,21,28,35,42,49,56,63,70]
```

Ejercicio 15.

Define `palabraEnMayusculas :: String -> String` que convierta una cadena a mayúsculas, usando `map toUpper`.

```
ghci> palabraEnMayusculas "Haskell"
"HASKELL"
```

Recuerda importar `Data.Char` *para usar* `toUpper`.

Ejercicio 16. ★

Define `sumaPositivos :: [Int] -> Int` que sume solo los elementos positivos de una lista, combinando `filter`, `sum` y una sección.

```
ghci> sumaPositivos [3,-1,4,-5,2]
9
ghci> sumaPositivos [-1,-2,-3]
0
```

Parte D — Inferencia y comprobación de tipos

Ejercicio 17.

Sin ejecutar GHCi, indica el tipo más general de cada una de las siguientes expresiones. Luego comprueba tu respuesta con `:t`.

a) `\x y -> x`

b) `\x y -> (x,y)`

c) `\f x -> f (f x)`

d) `map fst`

e) `filter (== 0)`

f) `zip [1..]`

Ejercicio 18.

Las siguientes expresiones contienen errores de tipos. Identifica el error en cada caso y explica por qué falla antes de comprobarlo en GHCi.

a) `[1, 2, True]`

b) `suma (3,4)` donde `suma :: Int -> Int -> Int`

c) `if 1 then "sí" else "no"`

d) `factorial True` donde `factorial :: Int -> Int`

e) `head [] :: Int`

Ejercicio 19. ★

Sin comprobarlo en GHCi, predice el tipo de las siguientes funciones ya escritas. Después cárgalas y compáralo con `:t`.

```haskell
f1 x = x == x

f2 x y = if x then y else y + 1

f3 xs = head xs : xs

f4 g x = g (g x)
```

Ejercicio 20.

Predice el tipo de las siguientes expresiones, formadas combinando `map`, `filter` y secciones ya vistas en esta sesión. Comprueba después con `:t`.

a) `aplicarDosVeces (map (*2))`

b) `\xs -> filter (>0) (map negate xs)`

c) `map (filter even)`